

Capability Negotiation & Version Matching, Worked Out by Hand

Capability negotiation is set algebra and version negotiation is a max-of-intersection — both worked out exactly on a concrete handshake.

What this document does. At connect-time, an MCP client and server each announce what they support, then agree on a common feature set and a common protocol version. This document treats both as exact computations: the usable feature set is an *intersection* of advertised capabilities, and the agreed version is the *maximum element of the intersection* of the two supported-version sets. We count the configuration space the sub-flags create and walk one full handshake number by number.

Concepts covered: capabilities as sets · the present/empty/absent ternary · sub-flags and the 2^k configuration space · effective-feature intersection · version sets and $\max(V_c \cap V_s)$ · why the handshake can fail.

Internal learning material. v1.0

1 The setup: one handshake, two capability lists

In `initialize`, each side sends a capabilities object. For our worked example:

Client $C = \{\text{sampling}\{\}, \text{roots}\{\text{listChanged}\}\}$, Server $S = \{\text{tools}\{\text{listChanged}\}, \text{resources}\{\text{listChan}$

Each entry is a *capability* (the key) optionally carrying *sub-flags* (the inner object). A feature works only if every side whose participation it requires has advertised it.

Intuition

Read capabilities as a contract, not a request. “**sampling**” in the client list means “*I, the client, am willing to run the LLM on a server’s behalf.*” “**tools**” in the server list means “*I expose callable tools.*” A feature is live only where the contract is signed on both required sides — exactly a set intersection over the participants a feature needs.

2 Step 1 — The present/empty/absent ternary

Each capability key is in one of three states, and they are *not* the same:

On the wire	Means	Sub-flags
"sampling": {"x":true}	supported, with sub-flags	set as given
"sampling": {}	supported, no sub-flags	all default off
(key omitted)	not supported	—

Watch out

"sampling": {} (supported) and "sampling": null are *not* interchangeable: most validators reject null. The classic bug is serialising a Python None into null when you meant the

empty object. “Supported with no options” is `{}`; “unsupported” is *absence*. Three states, two of which look superficially alike.

3 Step 2 — Sub-flags and the configuration space

A capability with k boolean sub-flags has 2^k possible configurations (each flag on or off), plus one more state for the whole capability being absent. So a single key with k sub-flags has $2^k + 1$ wire-distinguishable states.

For our server’s capabilities:

$$\text{tools } (k=1): 2^1=2, \quad \text{resources } (k=2): 2^2=4, \quad \text{prompts } (k=0): 2^0=1.$$

The number of distinct *server* capability objects expressible over just these three keys (each either absent or present-in-some-config) is the product

$$(2^1 + 1)(2^2 + 1)(2^0 + 1) = 3 \cdot 5 \cdot 2 = \boxed{30}.$$

Mini-example — this operation in isolation

Resources carries two sub-flags, `listChanged` and `subscribe`. Its $2^2 = 4$ live configurations are the truth table

	listChanged	subscribe
cfg 00	0	0
cfg 01	0	1
cfg 10	1	0
cfg 11	1	1

Our server runs **cfg 11**: it both emits `list_changed` notifications and accepts per-resource `subscribe`. Each independent flag doubles the space — k flags, 2^k rows.

4 Step 3 — The effective feature set (the intersection)

A feature is *live* iff each side it depends on advertised it. Tabulate the dependency and intersect against C and S :

Feature	Needs	in C ?	in S ?	Live?
tools/call	server tools	—	✓	✓
resources/read	server resources	—	✓	✓
prompts/get	server prompts	—	✓	✓
sampling/createMessage	client sampling	✓	(server uses)	✓
roots/list	client roots	✓	(server uses)	✓
elicitation/create	client elicitation	absent	—	no

Five of six features are live. `elicitation` is dead: the client never advertised it, so any `elicitation/create` the server attempts returns `-32601 Method not found`. The effective set is the intersection of “what each feature needs” with “what was advertised.”

Intuition

Capability negotiation exists so neither side guesses. The server does not probe for

elicitation and handle a failure; it reads the client’s advertised set once and simply never calls what isn’t there. The intersection is computed at connect-time and frozen for the session — one negotiation, zero runtime surprises.

5 Step 4 — Version negotiation as $\max(V_c \cap V_s)$

MCP versions are date strings, totally ordered by date:

$$v_1 = 2024-11-05 < v_2 = 2025-03-26 < v_3 = 2025-06-18.$$

Each side supports a *set* of versions. The client proposes one version; the server accepts it or counters with one it supports; the client accepts the counter or terminates. The stable outcome is the largest version both support:

$$v^* = \max(V_c \cap V_s), \quad \text{handshake fails if } V_c \cap V_s = \emptyset.$$

Client supports V_c	Server supports V_s	$V_c \cap V_s$	$v^* = \max$
$\{v_1, v_2, v_3\}$	$\{v_1, v_2, v_3\}$	$\{v_1, v_2, v_3\}$	v_3 (2025-06-18)
$\{v_3\}$	$\{v_1, v_2\}$	$\emptyset$	fail
$\{v_2, v_3\}$	$\{v_1, v_2\}$	$\{v_2\}$	v_2 (2025-03-26)
$\{v_1, v_2, v_3\}$	$\{v_1\}$	$\{v_1\}$	v_1 (2024-11-05)

Mini-example — this operation in isolation

A client proposes a future 2099-01-01 that the server has never heard of. Then $V_c \cap V_s$ over *known* versions still works: the server counters with its highest, $v_3 = 2025-06-18$, and the client — which also supports v_3 — accepts. The proposal being unknown does not break the handshake; the intersection of *actually-supported* versions does the deciding.

Watch out

A handshake completing is not the same as the version you wanted. If you ship code that assumes 2025-06-18 structured **elicitation** but negotiate down to v_1 , those calls will not exist on the wire. Read back the negotiated v^* and gate features on it — the *floor* of what both sides offer, not the ceiling of what you hoped for.

6 The argument, at a glance

#	Idea	Formula	Value
1	States of one capability key	present / empty / absent	ternary
2	Configs of k sub-flags	2^k	resources: 4
2	Server objects over 3 keys	$\prod (2^{k_i} + 1)$	$3 \cdot 5 \cdot 2 = 30$
3	Live features	$\text{deps} \cap \text{advertised}$	5/6
4	Negotiated version	$\max(V_c \cap V_s)$	v_3
4	Handshake failure	$V_c \cap V_s = \emptyset$	—

7 Equation cheat-sheet

Configs of k sub-flags	2^k
Wire states of one key	$2^k + 1$ (incl. absent)
Capability objects over keys i	$\prod_i (2^{k_i} + 1)$
Live feature set	$\{f : \text{deps}(f) \subseteq C \cup S\}$
Negotiated version	$v^* = \max(V_c \cap V_s)$
Handshake fails iff	$V_c \cap V_s = \emptyset$

A Reference implementation

```
def live_features(client_caps, server_caps, deps):
    adv = set(client_caps) | set(server_caps)
    return [f for f, need in deps.items() if set(need) <= adv]

def negotiate_version(Vc, Vs):
    common = set(Vc) & set(Vs)
    return max(common) if common else None    # None = fail

def server_config_space(subflag_counts):
    prod = 1
    for k in subflag_counts:
        prod *= (2**k + 1)                    # +1 for "absent"
    return prod

print(server_config_space([1, 2, 0]))        # 30
print(negotiate_version({"2025-03-26", "2025-06-18"},
                        {"2024-11-05", "2025-03-26"}))    # 2025-03-26
```

— end of document —